

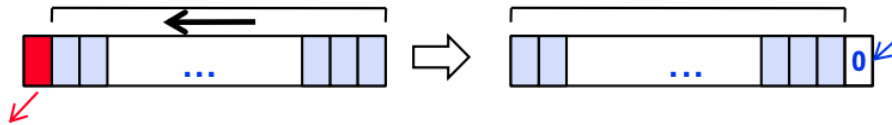
Lab 2

Operazioni Logiche
Operandi Allocati in Memoria (load/store)

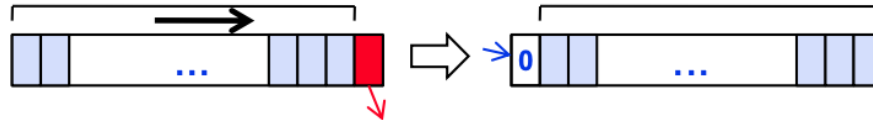
Operazioni logiche

Shift logico

A sinistra



A destra



Shift Left Logical

```
sll x9, x22, x19
```

$x9 = x22 \ll x19$

```
slli x9, x22, 5
```

$x9 = x22 \ll 5$

Shift Left Logical Immediate

Shift Right Logical

```
srl x9, x22, x19
```

$x9 = x22 \gg x19$

```
srli x9, x22, 5
```

$x9 = x22 \gg 5$

Shift Right Logical Immediate

Operazioni logiche

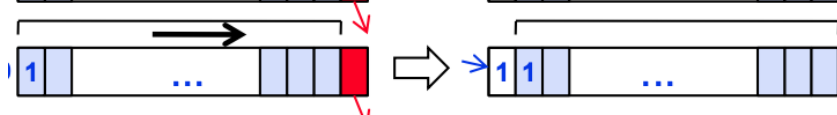
Shift aritmetico

A destra

Positivo



Negativo



A sinistra

Non esiste perché non ha senso: identico a `sll`

Shift Right Arithmetic

```
sra x9, x22, x19
```

$x9 = x22 \gg x19$

```
srai x9, x22, 5
```

$x9 = x22 \gg 5$

Shift Right Arithmetic Immediate

Operazioni logiche

- AND

```
and x9,x22,x19  
x9 = x22 & x19
```

```
andi x9,x22,5  
x9 = x22 & 5
```

- OR

```
or x9,x22,x19  
x9 = x22 | x19
```

```
ori x9,x22,5  
x9 = x22 | 5
```

- XOR

```
xor x9,x22,x19  
x9 = x22  $\oplus$  x19
```

```
xori x9,x22,5  
x9 = x22  $\oplus$  5
```

Pseudoistruzione

- NOT

```
not x5,x6  
x5 =  $\overline{x6}$ 
```

Operazioni logiche

- L'istruzione `and` permette di selezionare alcuni bit del primo operando indicandoli all'interno di una maschera (secondo operando)

`and x5, x6, x7`

x6	00100100 00010101 00001011 10100110	Sorgente
x7	00000100 00000110 00000010 00010010	Maschera di bit
x5	00000100 00000100 00000010 00000010	Risultato

Operazioni logiche

- L'istruzione `or` permette di ricopiare il primo operando, settando ad uno anche i bit che sono specificati nella maschera indicata come secondo operando

```
or x5, x6, x7
```

x6	00100100 00010101 00001011 10100110	Sorgente
x7	00000100 00000110 01000010 00010010	Maschera di bit
x5	00100100 00010111 01001011 10110110	Risultato

Esercizio 1 - Operazioni Logiche

Si supponga che i registri seguenti contengano i valori:

`x5 = 0xAAAAAAAA`, `x6 = 0x12345678`

Determinare il contenuto di **x7** dopo l'esecuzione delle seguenti istruzioni:

```
slli x7, x5, 4  
or   x7, x7, x6
```

- Supponendo che i registri x5 e x6 contengano i valori riportati sopra, determinare il contenuto di x7 dopo l'esecuzione di:

```
slli x7, x6, 4
```

- Supponendo che i registri x5 e x6 contengano i valori riportati sopra, determinare il contenuto di x7 dopo l'esecuzione di:

```
srli x7, x5, 3  
andi x7, x7, 0xFF
```

Esercizio 1 - Operazioni Logiche

Si supponga che i registri seguenti contengano i valori:

$x5 = 0xAAAAAAAA$, $x6 = 0x12345678$

- Determinare il contenuto di **x7** dopo l'esecuzione delle seguenti istruzioni:

```
slli x7, x5, 4  
or   x7, x7, x6
```

x7 = 0xbabefef8

- Supponendo che i registri x5 e x6 contengano i valori riportati sopra, determinare il contenuto di x7 dopo l'esecuzione di:

```
slli x7, x6, 4
```

x7 = 0x23456780

- Supponendo che i registri x5 e x6 contengano i valori riportati sopra, determinare il contenuto di x7 dopo l'esecuzione di:

```
srli x7, x5, 3  
andi x7, x7, 0xFF
```

x7 = 0x55

x5 = 0xAAAAAAAA, x6 = 0x12345678

x5 = 0x A A A A A A A A

x5 = 0b 1010 1010 1010 1010 1010 1010 1010 1010 1010

x5 = 0xAAAAAAAA, x6 = 0x12345678

x5 = 0x A A A A A A A A
x5 = 0b 1010 1010 1010 1010 1010 1010 1010 1010

slli x7, x5, 4

x7 = 0x A A A A A A A 0
x7 = 0b 1010 1010 1010 1010 1010 1010 1010 0000

x5 = 0xAAAAAAAA, x6 = 0x12345678

x5 = 0x A A A A A A A A
x5 = 0b 1010 1010 1010 1010 1010 1010 1010 1010

slli x7, x5, 4

x7 = 0x A A A A A A A 0
x7 = 0b 1010 1010 1010 1010 1010 1010 1010 1010 0000

x6 = 0x 1 2 3 4 5 6 7 8
x6 = 0b 0001 0010 0011 0100 0101 0110 0111 1000

x5 = 0xAAAAAAAA, x6 = 0x12345678

x5 = 0x A A A A A A A A
x5 = 0b 1010 1010 1010 1010 1010 1010 1010 1010

slli x7, x5, 4

x7 = 0x A A A A A A A 0
x7 = 0b 1010 1010 1010 1010 1010 1010 1010 1010 0000

x6 = 0x 1 2 3 4 5 6 7 8
x6 = 0b 0001 0010 0011 0100 0101 0110 0111 1000

or x7, x7, x6

x7 = 0b 1011 1010 1011 1110 1111 1110 1111 1000
x7 = 0x b a b e f e f 8

Esercizio 2 - Media di pari

Si scriva un programma in linguaggio RISC-V che carichi due numeri interi pari su **x5** e **x6** e calcoli il valore della loro media aritmetica. Il valore calcolato va inserito nel registro **x7**.

**In questo esercizio, utilizzare soltanto il set delle istruzioni "intere di base rv32i"
(usare un "shift" per fare la divisione per 2)**

Esercizio 2 - Media di pari

```
# calculate the average of the two even numbers in x5 and x6
# Put result in x7
.globl _start

.text
_start:

    # complete the code

exit:
addi x17, x0, 10  # call number 10 = exit
ecall
```

Operandi Allocati in Memoria

Operandi Allocati in Memoria

- Imparare a trasferire dati tra la memoria ed i registri in assembly
- Capire come funzionano le istruzioni "load" e "store"
- Capire il modello di memoria di un elaboratore RISC-V

Istruzione di trasferimento dati:

un comando che sposta i dati tra la memoria e i registri.

Indirizzo: un numero utilizzato per identificare la posizione di uno specifico elemento all'interno di una memoria, vista come un vettore unidimensionale.

Esempio: Load & Store word

Si scriva un programma in linguaggio RISC-V che carichi due numeri interi presenti nella memoria in word contigue e calcoli la loro somma. Il valore calcolato va salvato in una terza posizione della memoria contigua alle due usate per la somma.

- Per risolvere questo esercizio ci serve capire **dove in memoria** sono i nostri dati (**l'indirizzo**)

Esempio: Load & Store words

The screenshot shows a debugger window with the following components:

- Menu Bar:** File, Edit, Run, Settings, Tools, Help.
- Toolbar:** Icons for file operations and a status bar showing 'Run speed at max (no interaction)'.
- Settings Window (Open):**
 - General:**
 - ☒ Show Labels Window (symbol table)
 - ☒ Program arguments provided to program
 - ☒ Popup dialog for input syscalls (5, 6, 7, 8, 12)
 - ☒ Addresses displayed in hexadecimal
 - ☒ Values displayed in hexadecimal
 - Assembly:**
 - ☐ Assemble file upon opening
 - ☐ Assemble all files in directory
 - ☐ Assemble all files currently open
 - ☐ Assembler warnings are considered errors
 - ☐ Initialize Program Counter to global 'main' if defined
 - Execution:**
 - ☒ Permit extended (pseudo) instructions and formats
 - ☐ Self-modifying code
 - ☒ 64 bit
 - Editor...**
 - Highlighting...
 - Exception Handler...
 - Memory Configuration...** (selected)
- Memory Configuration...** (Tooltip): View and modify memory segment base addresses for the simulated processor.
- Messages / Run I/O:** A panel at the bottom with a 'Clear' button.
- Control and Status Registers:** A table on the right side of the window.

Control and Status Registers		
Floating Point		
Registers		
Name	Num...	Value
ustatus	0	0x00000...
fflags	1	0x00000...
frm	2	0x00000...
fcsr	3	0x00000...
uie	4	0x00000...
utvec	5	0x00000...
uscratch	64	0x00000...
uepc	65	0x00000...
ucause	66	0x00000...
utval	67	0x00000...
uip	68	0x00000...
cycle	3072	0x00000...
time	3073	0x00000...
instret	3074	0x00000...
cycleh	3206	0x00000...
timeh	3201	0x00000...
instreth	3202	0x00000...



Esempio: Load & Store words

Memory Configuration

Configuration

☒ Default
☐ Compact, Data at Address 0
☐ Compact, Text at Address 0

0xffffffff memory map limit address
0xffffffff kernel space high address
0xffff0000 MMIO base address
0x80000000 kernel space base address
0x7fffffff user space high address
0x7fffffff data segment limit address
0x7fffffc stack base address
0x7fffffc stack pointer (sp)
0x10040000 stack limit address
0x10040000 heap base address
0x10010000 .data base address
0x10008000 global pointer (gp)
0x10000000 data segment base address
0x10000000 .extern base address
0x0ffffffc text limit address
0x00400000 .text base address

Apply and Close

Apply

Cancel

Reset

Registers Floating Point Control and Status

Name	Number	Value
zero	0	0x0000000000000000
ra	1	0x0000000000000000
sp	2	0x000000007fffffc
gp	3	0x0000000010008000
tp	4	0x0000000000000000
t0	5	0x0000000000000000
t1	6	0x0000000000000000
t2	7	0x0000000000000000
s0	8	0x0000000000000000
s1	9	0x0000000000000000
a0	10	0x0000000000000000
a1	11	0x0000000000000000
a2	12	0x0000000000000000
a3	13	0x0000000000000000
a4	14	0x0000000000000000
a5	15	0x0000000000000000
a6	16	0x0000000000000000
a7	17	0x0000000000000000
s2	18	0x0000000000000000
s3	19	0x0000000000000000
s4	20	0x0000000000000000
s5	21	0x0000000000000000
s6	22	0x0000000000000000
s7	23	0x0000000000000000
s8	24	0x0000000000000000
s9	25	0x0000000000000000
s10	26	0x0000000000000000
s11	27	0x0000000000000000
t3	28	0x0000000000000000
t4	29	0x0000000000000000
t5	30	0x0000000000000000
t6	31	0x0000000000000000
pc		0x0000000000400000



Esempio: Load & Store words (prima idea)

```
.globl _start
```

```
.data
```

```
    v1: .word 8
```

```
    v2: .word 9
```

```
    v3: .word 0
```

```
.text
```

```
_start:
```

```
    li    t0, 0x10010000    # base address of our .data segment
```

```
    lw    t1, 0(t0)         # copy the first word to t1
```

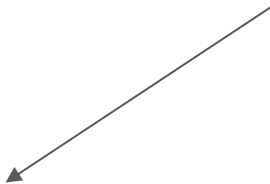
```
    lw    t2, 4(t0)         # copy the second word to t2
```

```
    add   t3, t1, t2        # sum the words and save it on register t3
```

```
    sw    t3, 8(t0)         # store the sum in the 3rd word starting from address in t0
```

```
...
```

**Indirizzo "hard-coded"?
Come evitarlo?**



Esempio: Load & Store words (prima idea)

Usare il simulatore per individuare i dati in memoria e nei registri dopo ogni istruzione

The screenshot displays a MIPS simulator interface with the following components:

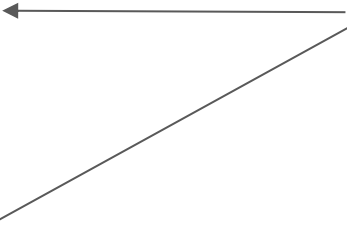
- Text Segment:** A table of assembly instructions with columns for Bkpt, Address, Code, Basic, and Source. The instructions include `li x8, 0x10010000`, `lw x5, 0(x8)`, `lw x6, 4(x8)`, `add x7, x5, x6`, `sw x7, 8(x8)`, `add a0, x0, x7`, `addi a7, x0, 93`, and `ecall`.
- Labels:** A table showing labels `(global)`, `start`, `4.a_somma_numeri.asm`, and `exit` with their corresponding addresses.
- Registers:** A table listing registers from `zero` to `pc` with their names, numbers, and values. Registers `t0` through `t7` are highlighted with a red box.
- Data Segment:** A table showing memory addresses and their values for different word sizes (4, 8, 16, 32 bits). A red box highlights the first four columns (Address, Value (+0), Value (+4), Value (+8), Value (+c)).
- Messages:** A log showing the message `-- program is finished running (17) --` repeated three times.

Esempio: Load & Store words (usando etichette)

```
.globl _start
.data
    v1: .word 8
    v2: .word 9
    v3: .word 0
.text
_start:
    la    t0, v1
    lw    t1, 0(t0)
    lw    t2, 4(t0)
    add   t3, t1, t2
    sw    t3, 8(t0)
```

...

Le etichette sono gli offset per arrivare agli indirizzi!



- **la** → **load address** è una pseudo-istruzione
- **Verificare nel simulatore RARS come viene tradotta l'istruzione**

P.S: Parleremo di "auipc" anche in altro lab

Esempio: Load & Store words (usando etichette)

```
.globl _start
```

```
.data
```

```
    v1: .word 8
```

```
    v2: .word 9
```

```
    v3: .word 0
```

```
.text
```

```
_start:
```

```
    la    t0, v1
```

```
    lw    t1, 1(t0)
```

```
    lw    t2, 4(t0)
```

```
    add   t3, t1, t2
```

```
    sw    t3, 8(t0)
```

```
...
```

Address + offset → lw?

Provare su RARS: `lw t1, 1(t0)`

Esempio: Load & Store words (usando etichette)

```
Messages Run I/O
Assemble: assembling /home/idrago/academic/teaching/2022/arch_i/labs/lab2/1_sum_numbers.asm
Assemble: operation completed successfully.
Assemble: assembling /home/idrago/academic/teaching/2022/arch_i/labs/lab4/3.a_sum_mem.asm
Assemble: operation completed successfully.
Error in /home/idrago/academic/teaching/2022/arch_i/labs/lab4/3.a_sum_mem.asm line 12: Runtime exception at 0x00400008: Load address not aligned to word boundary 0x100100
Step: execution terminated with errors.
```

```
.text
_start:
    la    t0, v1
    lw    t1, 1(t0)
    lw    t2, 4(t0)
    add   t3, t1, t2
    sw    t3, 8(t0)
```

Address + offset → lw → word aligned

Vincolo di allineamento: requisito per cui, in molte architetture, le parole devono iniziare sempre a indirizzi multipli di 4 e le parole doppie a indirizzi multipli di 8.

Approfondimento. In molte architetture, le parole devono iniziare sempre a indirizzi multipli di 4 e le parole doppie a indirizzi multipli di 8. Questo requisito si chiama **vincolo di allineamento** ed è comune a molte altre architetture (nel Capitolo 4 viene suggerito come l'allineamento produca una velocità maggiore nel trasferimento dei dati). I RISC-V e gli Intel x86 non hanno il vincolo di allineamento, mentre i MIPS sì.

Esercizio 3 - Media interi

Si scriva un programma in linguaggio RISC-V che carichi 4 numeri interi presente nella memoria in word contigue e calcoli il valore intero della loro media aritmetica (arrotondamento per difetto). Il valore calcolato va salvato in un'ulteriore posizione della memoria contigua a quelle usate per il calcolo.

In questo esercizio, utilizzare soltanto il set delle istruzioni "intere di base rv32i".

Esercizio 3 - Media interi

```
.globl _start
.data
    numbers: .word 12, 16, 20, 24    # Four sample integers
    result:  .word 0                 # Space for result

.text
_start:
    la t0, numbers    # Load address of first number

    # complete the code

    # Exit program
    li a7, 10          # System call for exit
    ecall
```

Esercizio 4 - Somma dei quadrati

Implementa un programma in linguaggio RISC-V che prenda in input due numeri interi da memoria e calcoli la somma dei quadrati di entrambi i numeri. Il risultato deve essere salvato in un'ulteriore posizione di memoria contigua.

In questo esercizio, utilizzare l'istruzione "mul" - controllare il suo formato nel "help"

Esercizio 4 - Somma dei quadrati

```
.globl _start  
.data  
    num1:      .word 5          # First number  
    num2:      .word 7          # Second number  
    result:    .word 0          # Location for result  
  
.text  
_start:  
  
    # complete the code  
  
    # Exit program  
    li a7, 10                  # System call for exit  
    ecall
```

Esercizio 5 - Signed vs Unsigned

Si scriva un programma in linguaggio RISC-V che calcoli la somma degli elementi di un array con 3 elementi (byte) in due modi:

1. Considerando i valori come numeri con segno (signed byte)
2. Considerando i valori come numeri senza segno (unsigned byte)

I risultati vanno salvati nelle variabili "result1" e "result2".

Esercizio 5 - Signed vs Unsigned

```
.globl _start
.data
    array:    .byte 130, 150, 200    # Array with 3 bytes
    result1:  .word 0                # result signed
    result2:  .word 0                # result unsigned

.text
_start:
```

```
# complete the code
```

```
# Exit program
li a7, 10          # System call for exit
ecall
```

Esercizio 6 - Shift right

Si scriva un programma in linguaggio RISC-V che:

1. Legga il valore di una word in memoria (var1) in un registro
2. Estragga i singoli byte di questa word utilizzando shift e operazioni logiche
3. Salvi ciascun byte estratto nelle variabili byte1 (byte meno significativo), byte2, byte3 e byte4 (byte più significativo)

Suggerimenti:

- Usare srli per spostare il byte desiderato nella posizione meno significativa
- Usare andi con la maschera 0xFF per isolare il byte

Esercizio 6 - Shift right

```
.globl _start
.data
    var1:  .word 0x12345678    # Word value to process
    byte4: .byte 0             # Most significant byte (will be 0x12)
    byte3: .byte 0             # Third byte (will be 0x34)
    byte2: .byte 0             # Second byte (will be 0x56)
    byte1: .byte 0             # Least significant byte (will be 0x78)
.text
_start:

    # complete the code

    # Exit program
    li a7, 10                  # System call for exit
    ecall
```


Esercizio 7 - Estrarre Bit

Determinare una sequenza di istruzioni RISC-V che consenta di "estrarre" i bit da **b2 a b3** dal registro **x5** e li sostituisca ai bit da **b6 a b7** del registro **x6** senza modificare gli altri bit del registro **x5** e **x6**.

Assicuratevi di verificare il vostro codice utilizzando

`x5 = 0x00000000` e `x6 = 0xFFFFFFFF`

Questo potrebbe rivelare una svista comune.

Esercizio 7 - Estrarre Bit

x5 = 0x ... 0 0 0
x5 = 0b ... 0000 0000 **00**00

bit 2-3

x6 = 0x ... f f f
x6 = 0b ... 1111 **11**11 1111

bit 6-7

Esercizio 7 - Estrarre Bit

x5 = 0x ... 0 0 0 0
x5 = 0b ... 0000 0000 0000 **00**00 **bit 2-3**

x6 = 0x ... F F F F **bit 6-7**
x6 = 0b ... 1111 1111 **11**11 1111

Vogliamo ottenere:

x6 = 0x ... F F 3 F
x6 = 0b ... 1111 1111 **00**11 1111

Esercizio 7 - Estrarre Bit

Determinare una sequenza di istruzioni RISC-V che consenta di estrarre i bit da **b11 a b16** dal registro **x5** e li sostituisca ai bit da **b26 a b31** del registro **x6** senza modificare gli altri bit del registro **x5** e **x6**.

Assicuratevi di verificare il vostro codice utilizzando

`x5 = 0x00000000` e `x6 = 0xFFFFFFFF`

Questo potrebbe rivelare una svista comune.